파이썬 완전 백과사전 2권 · 딥러닝 기초 교안

3일차

데이터를 다듬어 나르고 외우지 않고 배우게 하기

11~15차시 · 책 7장 7.7절 ~ 8장 학습 잘 시키기

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/07-data-pipeline, 08-training-well

오늘의 시간표

차시	주제	책 범위
11	길이 맞추기, 나누기, 불균형, 일꾼	7.7 ~ 7.11
12	과적합 진단, weight decay, 드롭아웃	8장 들어가며 ~ 8.3
13	배치 정규화, 초기화, 스케줄러	8.4 ~ 8.7
14	조기 종료, 클리핑, 재현성, 디버깅	8.8 ~ 8.11
15	대책을 하나씩 더하는 실습, 마무리	8.12 ~ 8.13

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분

준비: examples/vol2 폴더에서 uv sync · 7장 실습(b23)은 24초, 8장 실습(b25)은 33초가 걸린다

11차시

11

길이 맞추기, 나누기, 불균형, 일꾼

책 7.7절 ~ 7.11절

학습 목표

- 길이가 다른 샘플을 collate_fn으로 채워 한 배치로 묶는다
- 씨앗을 고정해 나누고, 증강은 학습용에만 건다
- 드문 클래스는 더 자주 뽑고, 일꾼 프로세스로 빨리 나른다

7.7 문장은 길이가 제각각이다

```
# 예제용으로 만든 짧은 식당 리뷰와 라벨(1: 좋음, 0: 나쁨)
리뷰들 = [("맛있어요", 1), ("너무 짜요", 0), ("또 오고 싶어요", 1), ...]
def __getitem__(self, i: int) -> tuple[torch.Tensor, int]:
    문장, 라벨 = self.자료[i]        # 글자마다 번호(1부터, 0은 비워 둠)
    return torch.tensor([self.글자_번호[글자] for 글자 in 문장]), 라벨

next(iter(DataLoader(데이터, batch_size=4)))
```

```
맛있어요 → [13, 22, 19, 21], 라벨 1
너무 짜요 → [7, 14, 1, 25, 21], 라벨 0
또 오고 싶어요 → [10, 1, 20, 3, 1, 17, 19, 21], 라벨 1
오류: stack expects each tensor to be equal size, but got [4] at entry 0 and [5] at entry 1
```

기본 규칙은 샘플을 **torch.stack**으로 쌓는다. 모양이 같아야 쌓이니 "0번은 [4], 1번은 [5]"에서 멈춘다

7.7 빈칸 번호 0으로 채워 길이 맞추기

```
def 패딩_묶기(배치):
    문장들 = [문장 for 문장, _ in 배치]
    return {
        "입력": pad_sequence(문장들, batch_first=True,
                             padding_value=0),
        "길이": torch.tensor([len(문장) for 문장 in 문장들]),
        "정답": torch.tensor([라벨 for _, 라벨 in 배치]),
    }

로더 = DataLoader(샘플들, batch_size=2,
                  collate_fn=패딩_묶기)
```

```
tensor([[13, 22, 19, 21,  0,  0,  0,  0],
        [ 7, 14,  1, 25, 21,  0,  0,  0],
        [10,  1, 20,  3,  1, 17, 19, 21]])
길이: [4, 5, 8] 정답: [1, 0, 1]
진짜 글자 자리(마스크):
tensor([[1, 1, 1, 1, 0, 0, 0, 0],
        [1, 1, 1, 1, 1, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 1]], ...)
(2, 5) [4, 5]
(1, 8) [8]
```

collate_fn은 샘플 목록을 받아 배치 하나를 돌려주는 평범한 함수. 길이와 마스크(입력 != 0)를 함께 담고, 채우기는 **배치 안에서만**

7.7 비슷한 길이끼리 묶으면 헛일이 준다

그냥 섞어서 32개씩

46.1%

배치 칸 가운데 0으로 채운 칸

길이순으로 32개씩 끊고 묶음만 섞기

2.6%

배치 수는 둘 다 32개

```
길이순 = sorted(range(len(문장들)), key=lambda i: len(문장들[i]))
묶음들 = [길이순[i:i + 32] for i in range(0, len(길이순), 32)]
묶음들 = [묶음들[i] for i in torch.randperm(len(묶음들), generator=g).tolist()]
비슷한_길이끼리 = DataLoader(문장들, batch_sampler=묶음들, collate_fn=패딩_묶기)
```

7.8 random_split과 씨앗

```
데이터 = TensorDataset(torch.arange(100).float(), torch.arange(100))  # 샘플 100개
학습, 검증 = random_split(데이터, [0.8, 0.2],
                          generator=torch.Generator().manual_seed(42))
학, 검, 시 = random_split(데이터, [70, 15, 15], generator=...)  # 개수로, 세 갈래로
```

```
크기: 80 20 Subset
검증에 뽑힌 번호(앞 8개): [2, 91, 8, 53, 30, 90, 26, 23]
검증 2번 샘플: (tensor(8.), tensor(8)) = 원래 데이터의 8 번
같은 씨앗이면 같은 분할: True
학습과 검증이 겹치나: False
세 갈래: 70 15 15
```

- Subset은 복사본이 아니라 원래 데이터를 들여다보는 창
- 씨앗이 없으면 어제와 오늘 **다른 시험지**로 잰다

7.8 transform이 다를 때의 함정

잘못: 하나를 나누면

```
학습, 검증 = random_split(
    숫자_데이터셋(transform=흔들기),
    [0.7, 0.3], generator=g)
```

```
검증 0번을 두 번 꺼내면
9.1541 8.9707
```

검증 쪽에도 증강이 걸려 잴 때마다 흔들린다

바름: 같은 번호로 두 Subset

```
번호 = torch.randperm(10, generator=g).tolist()
학습 = Subset(숫자_데이터셋(transform=흔들기),
              번호[:7])
검증 = Subset(숫자_데이터셋(transform=None),
              번호[7:])
```

```
검증 0번: 0.0 0.0
학습 0번: 2.7821 3.0568
```

7.9 드문 클래스를 더 자주 뽑기

```
클래스별_개수 = torch.bincount(정답)             # [950, 50]
샘플_가중치 = 1.0 / 클래스별_개수[정답].float()  # 드문 클래스일수록 크게
샘플러 = WeightedRandomSampler(샘플_가중치, num_samples=len(데이터),
                               replacement=True, generator=g)
DataLoader(데이터, batch_size=50, sampler=샘플러)
```

한 에폭(1,000번 뽑기)	**그냥 섞기**	**WeightedRandomSampler**
정상 : 불량	950 : 50	**500 : 500**
앞 8배치의 불량 개수	2, 4, 2, 2, 2, 4, 5, 3	27, 25, 24, 22, 29, 26, 23, 23
서로 다른 불량 샘플	50종	**여전히 50종**

가중치 정상 0.00105, 불량 0.02000 · 같은 50개를 평균 10번씩 반복하니 **증강과 함께**, **학습용 로더에만**

7.10 일꾼 프로세스와 __main__ 가드

가드_없음.py · 종료 코드 1

```
로더 = DataLoader(TensorDataset(torch.arange(8)),
          batch_size=4, num_workers=2)
for (배치,) in 로더:
    print("배치:", 배치.tolist())
```

```
An attempt has been made to start
a new process before the current
process has finished its
bootstrapping phase.
```

가드_있음.py · 종료 코드 0

```
def main() -> None:
    로더 = DataLoader(..., num_workers=2)
    for (배치,) in 로더:
        print("배치:", 배치.tolist())

if __name__ == "__main__":
    main()
```

```
배치: [0, 1, 2, 3]
배치: [4, 5, 6, 7]
```

macOS·Windows의 일꾼은 우리 파일을 **처음부터 다시 import** 한다(spawn). num_workers를 쓰면 언제나 가드 안에서

7.10 일꾼이 몇이면 빨라질까

샘플 하나 읽기 10ms, 배치 하나 학습 50ms로 흉내 내고(160장, 배치 16) 다음 배치를 **기다린 시간**을 쟀다

num_workers	첫 에폭	둘째 에폭	데이터 대기
0	2.5초	2.4초	**78%**
2	1.6초	1.1초	50%
4	1.3초	**0.7초**	27%

IterableDataset + 일꾼 2명

```
나눠_읽기=False:
[0, 0, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5]
나눠_읽기=True:
[0, 1, 2, 3, 4, 5]
```

몫은 get_worker_info()로 직접 나눈다

첫 에폭이 느린 것은 일꾼을 띄우는 비용 · persistent_workers=True로 살려 두기 · 대기 비율을 재며 늘린다

7.11 도형 그림 파이프라인 한 바퀴

일부러 불균형하게: 동그라미 120장, 네모 60장, 세모 20장 (32×32 예제용 그림)

1. 씨앗 42로 학습 80%, 검증 20%
2. 학습용 그림에서만 채널 평균·표준편차
3. 학습용은 증강 + 정규화, 검증용은 정규화만
4. 학습 로더에 WeightedRandomSampler, 둘 다 일꾼 2명
5. 한 에폭 돌며 모양·값·클래스 개수 확인

```
학습용 채널 평균: [0.841, 0.842, 0.845]
학습용 클래스별 개수: [98, 46, 16]
클래스: ['circle', 'square', 'triangle']
  | 배치 수: 5 1
첫 배치: (32, 3, 32, 32)
  torch.float32 torch.int64
첫 배치 평균 0.089, 표준편차 0.959
한 에폭 동안 본 클래스별 개수: [50, 52, 58]
검증 데이터: (40, 3, 32, 32)
  | 두 번 읽어도 같은가: True
```

11차시 · 7.11 실습: 도형 그림 파이프라인 한 바퀴 · 예제 07-data-pipeline/b23_practice.py, 도형_만들기.py (24초, 긴 줄은 화면에서 접음)

직접 실행 10분

1. b23_practice.py를 실행하고, 출력 일곱 줄을 7.11절의 다섯 단계에 하나씩 짝지어 본다
2. 복사본의 학습_변환에서 RandomCrop 다음에 v2.RandomRotation(15)를 더한다. 첫 배치의 평균과 표준편차는 어떻게 바뀌나?
3. 학습 로더의 sampler=샘플러를 shuffle=True로 바꾼다. 한 에폭 동안 본 클래스별 개수는?

```
cd examples/vol2/07-data-pipeline && uv run python b23_practice.py
```

검증용 DataLoader에 넣으면 안 되는 것은 무엇일까요?

Q

가 학습 데이터로 구한 값으로 Normalize

나 WeightedRandomSampler

다 shuffle=False

라 num_workers=2

11차시 정리

길이가 다른 샘플은 **collate_fn**에서 pad_sequence로 채우고 길이·마스크를 함께. 비슷한 길이끼리 묶으면 패딩이 46.1%에서 2.6%로

나누기는 **씨앗 고정 generator**로. transform이 다르면 같은 번호로 Subset 두 개. 불균형은 **WeightedRandomSampler**, 학습용에만

num_workers는 __main__ 가드 안에서, 기다린 시간을 재며 늘린다

다음 차시: 8장 학습 잘 시키기 · 학습 곡선으로 과적합 진단하기

12차시

12

과적합 진단, weight decay, 드롭아웃

책 8장 들어가며 ~ 8.3절

학습 목표

- 학습·검증 손실 곡선으로 과적합과 과소적합을 가른다
- weight decay로 큰 가중치에 벌점을 주고, Adam에는 AdamW를 쓴다
- 드롭아웃이 학습 때와 평가 때 다르게 움직이는 것을 확인한다

8장 문제집 답을 통째로 외운 학생

외우는 공부

"3번 답은 ②, 4번 답은 ⑤"

문제집은 100점, 숫자 하나 바뀐 시험은 손을 못 댄다. 해답지 오타까지 외운다

실험 재료

손글씨 숫자 8×8, 1,797장

학습 **200** · 검증 500 · 테스트 1,097
입력 64 → 은닉 256 → 256 → 10

해답지에 오타

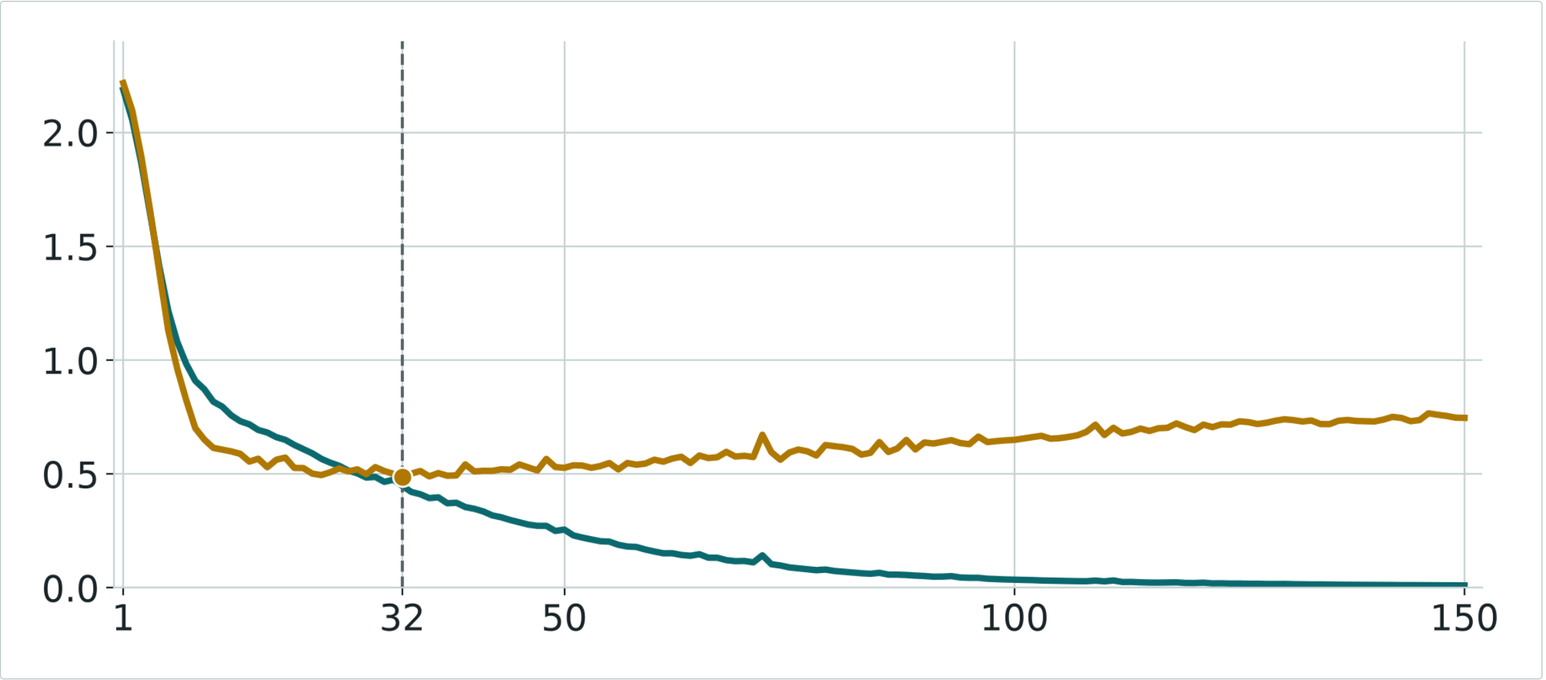
학습 정답의 20%를 무작위로

실제로 틀린 정답 **34개**. 검증·테스트 정답은 그대로

```
from 학습도구 import 데이터_준비, 로더_만들기, 모델_만들기, 평가, 한_에폭_학습
```

12차시 · 8장 들어가며, 8.1 이 장의 실험 재료 · 예제 08-training-well/학습도구.py

8.1 학습 곡선 그리기



— 학습 손실 · — 검증 손실 · 가로는 에폭 · 점선 = 검증 손실 최저

검증 손실 최저

32에폭 0.4883

100에폭부터 학습 정확도

100%

검증 정확도 30 → 150에폭

87.2% → 82.0%

학습 손실은 계속 줄고 검증 손실은 **바닥을 찍고 오른다**: 과적합의 전형적인 신호

8.1 과소적합은 어떻게 생겼을까

같은 150에폭, 설정만 바꿔서	학습 손실 (정확도)	검증 손실 (정확도)
은닉 2칸짜리 작은 모델	1.975 (25.0%)	1.989 (21.0%)
학습률 1e-5 (너무 작음)	2.059 (48.5%)	2.088 (42.4%)
은닉 256칸, 학습률 1e-3	0.010 (100.0%)	0.746 (82.0%)

학습 손실	검증 손실	진단	먼저 해 볼 일
높음	높음	과소적합	모델 키우기, 더 오래, 학습률 조정
낮음	높음(점점 오름)	과적합	데이터·증강, 정규화, 조기 종료
낮음	낮음(비슷)	좋음	테스트 데이터로 마지막 확인

순서: 먼저 **과적합할 만큼** 학습 손실을 내리고, 그다음 정규화로 검증 손실을 끌어내린다

8.2 큰 가중치에 벌점 매기기

```
# 방법 1: 손실에 (모든 가중치 제곱의 합) × 벌점_세기 / 2를 직접 더한다
벌점 = sum((p ** 2).sum() for p in 모델1.parameters())
손실1 = nn.functional.mse_loss(모델1(X), y) + 벌점_세기 / 2 * 벌점

# 방법 2: 옵티마이저의 weight_decay 옵션에 맡긴다
옵티마이저2 = torch.optim.SGD(모델2.parameters(), lr=0.1, weight_decay=벌점_세기)
```

```
방법 1 가중치: tensor([[ 0.1750, -0.1404,  0.0195]])
방법 2 가중치: tensor([[ 0.1750, -0.1404,  0.0195]])
같은가? True
```

Adam에는 AdamW

벌점을 그래디언트에 섞지 않고 매 걸음 학습률 × weight_decay만큼 **따로** 줄인다

8.2 벌점은 적당한 세기가 있다

```
옵티마이저 = torch.optim.AdamW(모델.parameters(), lr=1e-3, weight_decay=wd)
```

weight_decay	가중치 크기	학습 손실	검증 손실	검증 정확도
0.0	24.85	0.010	0.746	82.0%
0.01	24.82	0.011	0.738	82.6%
0.1	23.80	0.015	0.693	82.6%
1.0	17.95	0.111	**0.498**	85.0%
3.0	12.38	0.493	0.542	**89.6%**
10.0	7.40	1.166	1.096	77.8%

- 세질수록 가중치가 작아진다
- 0.01, 0.1은 효과가 미미
- 1.0~3.0에서 검증이 좋아지고
- 10.0은 둘 다 높다: 과소적합

8.3 드롭아웃: 매번 몇 명씩 빠지는 팀

회의 때마다 무작위로 두세 명이 빠지면, 누가 빠져도 일이 돌아가게 모두가 조금씩 다 할 줄 알게 된다

학습 때 은닉 칸 일부를 매번 0으로 끈다

```
드롭아웃 = nn.Dropout(p=0.5)   # 칸마다 50% 확률로 끈다
x = torch.ones(10)
드롭아웃.train()  # 학습 모드에서 두 번, 평가 모드에서 두 번
```

```
학습 모드 1번째: tensor([0., 0., 2., 0., 0., 0., 2., 2., 0., 2.])
학습 모드 2번째: tensor([2., 2., 2., 0., 2., 2., 2., 2., 0., 0.])
평가 모드 1번째: tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
평가 모드 2번째: tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
학습 모드 평균: 0.9985    # 숫자 10만 개
```

살아남은 칸은 1 / (1 - p) = 2배로 키워 평균을 맞추고, 평가 때는 그대로 통과

8.3 eval()을 깜빡하면

```
모델 = 모델_만들기(드롭아웃=0.5)     # 150에폭 학습한 뒤
숫자 = X_검증[:1]                    # 검증 데이터의 첫 숫자 하나를 세 번
모델.train()                         # eval()을 깜빡했다면
```

train 모드로 재면

확신도 [0.9913, 0.9398, 0.7024]

82.8%

eval 모드로 재면

확신도 [0.9911, 0.9911, 0.9911]

87.6%

오류 없이 성능을 **실제보다 낮게, 실행할 때마다 다르게** 잰다 · 아래 큰 숫자는 검증 정확도

12차시 · 8.3 학습 때와 평가 때가 다릅니다 · 예제 08-training-well/b06_dropout_eval.py (핵심 부분, 3초)

8.3 얼마나 꺼야 할까

드롭아웃 p	학습 손실	학습 정확도	검증 손실	검증 정확도
0.0	0.010	100.0%	0.746	82.0%
0.2	0.019	100.0%	0.629	83.0%
0.5	0.092	97.0%	0.452	87.6%
0.7	0.292	89.0%	**0.401**	**90.8%**

- 학습 성적은 내려가고 검증 성적은 오른다
- 100% 아래로 = 틀린 정답을 다 외우지 못한다

보통은 0.1~0.5에서 시작해 칸이 많은 은닉층 뒤에. 0.7까지 좋아진 것은 데이터가 아주 적고 지저분해서

직접 실행 10분

1. b01_curve.py를 실행하고 learning_curve.png를 열어, 검증 손실이 가장 낮은 에폭을 그래프에서 찾는다
2. 복사본에서 첫 데이터_준비()를 데이터_준비(잡음_비율=0.0)으로 바꾼다. 검증 손실 최저 에폭과 150에폭 검증 정확도는?
3. b07_dropout_p.py 복사본의 p 목록에 0.9를 더한다. 학습·검증 정확도는 어떻게 되나?

```
cd examples/vol2/08-training-well && uv run python b01_curve.py
```

12차시 · 직접 실행 · b01은 3초, b07은 5초 · 복사본은 08-training-well 폴더 안에 두어야 학습도구를 import한다

학습 손실 0.010, 검증 손실 0.746. 먼저 해 볼 일이 아닌 것은?

Q

가 AdamW에 weight_decay 주기

나 은닉층 뒤에 드롭아웃 넣기

다 은닉 칸을 늘려 모델 키우기

라 학습 데이터에 증강 걸기

12차시 정리

학습 손실과 검증 손실을 **함께 그린다**. 학습만 내려가고 검증이 오르면 과적합, 둘 다 높으면 과소적합

weight decay는 큰 가중치에 벌점. Adam 계열은 AdamW, 세기는 검증 데이터로 고른다

드롭아웃은 학습 때만 칸을 끄고 남은 칸을 1/(1-p)배. 평가 전에는 꼭 eval()

다음 차시: 배치 정규화, 가중치 초기화, 학습률 스케줄러

13차시

13

배치 정규화, 초기화, 스케줄러

책 8.4절 ~ 8.7절

학습 목표

- 배치 정규화가 학습 때와 평가 때 하는 일을 손으로 확인한다
- 가중치의 처음 값이 깊은 신경망을 살리고 죽이는 것을 본다
- 스케줄러로 학습률을 에폭에 따라 바꾼다

8.4 배치 정규화: 반마다 다른 난이도 맞추기

1반 평균 90점, 2반 평균 50점이면 "반 평균에서 얼마나 떨어졌나"로 바꿔 비교한다

```
x = [[1, 100], [2, 300],
     [3, 200], [4, 400]]
bn = nn.BatchNorm1d(2)
```

```
처음 running_mean: tensor([0., 0.])
학습 모드 출력:
tensor([[-1.3416, -1.3416],
        [-0.4472,  0.4472],
        [ 0.4472, -0.4472],
        [ 1.3416,  1.3416]])
손으로 계산과 같은가? True
한 번 본 뒤 running_mean: tensor([ 0.2500, 25.0000])
평가 모드 출력(첫 줄): tensor([[0.7262, 1.8366]])
오류: Expected more than 1 value per channel
when training, got input size torch.Size([1, 2])
```

학습 모드는 **지금 배치**의 평균·분산으로, 평가 모드는 **기록해 둔 이동 평균**으로 표준화한다

13차시 · 8.4 배치 정규화 · 예제 08-training-well/b08_bn.py (핵심 부분, 마지막 오류 줄은 화면에서 접음)

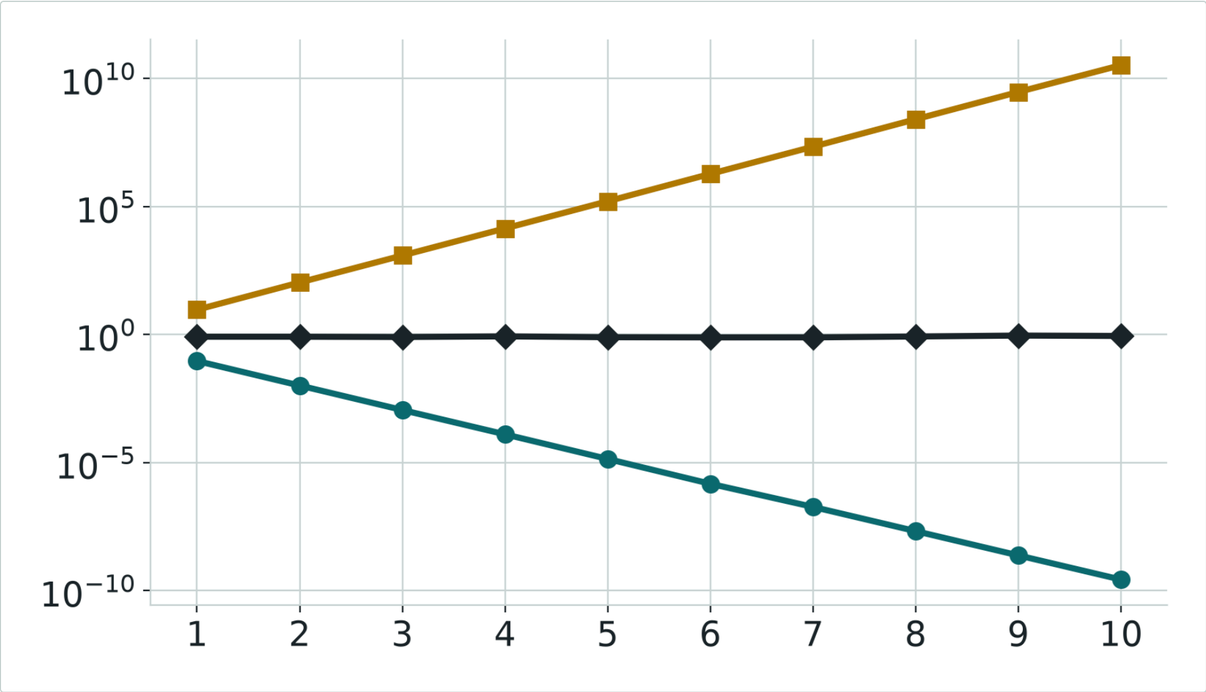
8.4 깊은 신경망이 살아난다

```
(X_학습, y_학습), (X_검증, y_검증), _ = 데이터_준비(잡음_비율=0.0, 학습_개수=1000)
모델 = 깊은_모델(배치정규화=배치정규화)      # 폭 64짜리 은닉층 8개
옵티마이저 = torch.optim.SGD(모델.parameters(), lr=0.05)
```

검증 정확도	1에폭	3에폭	5에폭	10에폭	20에폭
배치 정규화 없음	9%	9%	9%	10%	10%
배치 정규화 있음	**60%**	86%	82%	91%	**96%**

> **8.5 데이터 증강은 7장에서** 과적합을 막는 가장 확실한 길은 데이터를 늘리는 것. 증강은 7.6절 그대로, 학습 데이터에만. 효과는 15차시 실습에서 숫자로

8.6 첫 단추가 잘못 끼워지면



● std 0.01 · ■ std 1.0 · ◆ Kaiming · 가로 층 · 세로 출력 표준편차

10층 · std 0.01
2.71e-10

10층 · std 1.0
3.34e+10

10층 · Kaiming $\sqrt{(2/n)}$
8.87e-01

13차시 · 8.6 가중치 초기화 · 책의 b10_init_std.py와 같은 입력·층·초기화로 다시 그린 그림

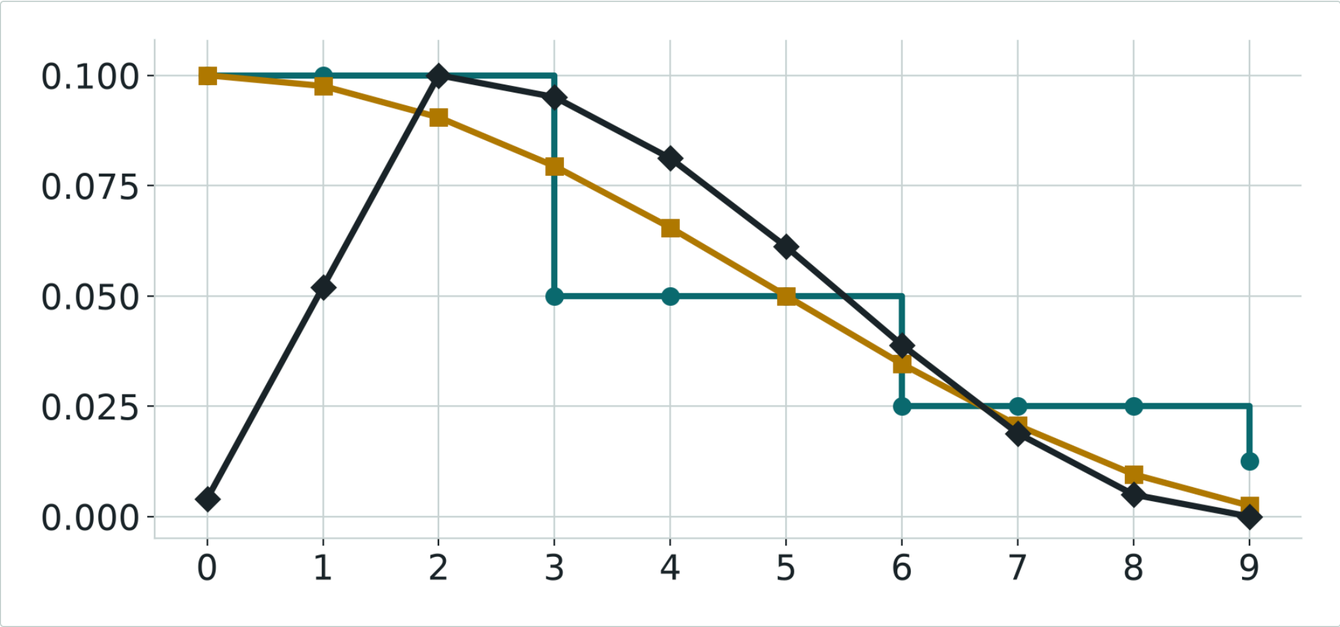
8.6 나쁜 초기화로 학습이 멈추는 모습

```
for 층 in 모델.modules():                # 모델 안의 nn.Linear만 골라 다시 채운다
    if isinstance(층, nn.Linear):
        nn.init.kaiming_normal_(층.weight, nonlinearity="relu")
```

깊은 모델, 20에폭	검증 손실	정확도
모두 0	2.3053	9.8%
표준편차 0.01	2.3053	9.8%
PyTorch 기본값	2.3052	9.8%
Kaiming	**0.1364**	**96.4%**

- 2.305 ≈ ln(10): 아무것도 못 배움
- 모두 0이면 모든 칸이 똑같이 움직인다
- 기본값은 Kaiming보다 2.4배쯤 작다

8.7 처음엔 크게, 가까워지면 작게



● StepLR(3, 0.5) · ■ Cosine · ◆ OneCycle · 가로는 에폭, 세로는 학습률

StepLR 3에폭마다 절반, 계단

Cosine 0 근처까지 부드럽게, 요즘 가장 흔함

OneCycle 올렸다가(워밍업) 내림

optimizer.step() 다음에 scheduler.step() · OneCycle은 보통 배치마다, total_steps = 에폭 수 × 에폭당 배치 수

8.7 검증 손실을 보고 줄이기

```
스케줄러 = ReduceLROnPlateau(옵티마이저,
    mode="min", factor=0.5, patience=2)

# 예제용 검증 손실: 0.50 근처에서 멈춘다
for 에폭, 손실 in enumerate(검증_손실들, start=1):
    스케줄러.step(손실)   # 검증 손실을 넘겨준다
```

- 최고 기록을 **더 낮추지 못한** 에폭을 센다
- 두 번까지 참고, 세 번째에 절반으로
- 몇 에폭 돌릴지 정하기 어려울 때 편리

```
 1에폭 0.90 -> 0.1000
 2에폭 0.70 -> 0.1000
 3에폭 0.55 -> 0.1000
 4에폭 0.50 -> 0.1000
 5에폭 0.51 -> 0.1000
 6에폭 0.50 -> 0.1000
 7에폭 0.52 -> 0.0500
 8에폭 0.49 -> 0.0500
 9에폭 0.50 -> 0.0500
10에폭 0.51 -> 0.0500
11에폭 0.50 -> 0.0250
12에폭 0.50 -> 0.0250
```

13차시 · 8.7 검증 손실을 보고 줄이기 · 예제 08-training-well/b14_plateau.py (출력에서 "검증 손실", "학습률" 글자를 줄임)

직접 실행 10분

1. b09_bn_speed.py와 b11_init_train.py를 실행해, 깊은 모델을 살린 두 방법의 20에폭 검증 정확도를 나란히 적는다
2. b11 복사본의 초기화() 함수에 "표준편차 1.0" 방식(nn.init.normal_(층.weight, std=1.0))을 더하면?
3. b10_init_std.py 복사본에 Xavier(nn.init.xavier_normal_) 줄을 더한다. ReLU 층 10개를 지나면 표준편차는?

```
cd examples/vol2/08-training-well && uv run python b11_init_train.py
```

ReLU 층 8개 모델의 검증 손실이 20에폭 내내 2.305. 가장 먼저 의심할 것은?

Q

가 스케줄러를 쓰지 않았다

나 가중치의 처음 값

다 드롭아웃이 약하다

라 weight decay가 약하다

13차시 정리

배치 정규화는 배치 평균·분산으로 표준화하고, 평가 때는 기록해 둔 이동 평균을 쓴다. 깊은 모델을 살린다

초기화는 0으로 시작하지 않는다. ReLU에는 Kaiming, 시그모이드·tanh에는 Xavier

스케줄러는 optimizer.step() 다음에. 시간표대로(Step, Cosine, OneCycle) 또는 검증 손실을 보고(Plateau)

다음 차시: 조기 종료, 그래디언트 클리핑, 재현성, 손실이 안 줄 때 디버깅

14차시

14

조기 종료, 클리핑, 재현성, 디버깅

책 8.8절 ~ 8.11절

학습 목표

- 조기 종료를 직접 만들고 가장 좋았던 가중치로 되돌린다
- 클리핑으로 폭주를 막고, 같은 결과를 다시 얻게 고정한다
- 손실이 안 줄 때 싼 확인부터 차례로 원인을 좁힌다

8.8 가장 좋았던 순간에 멈추기

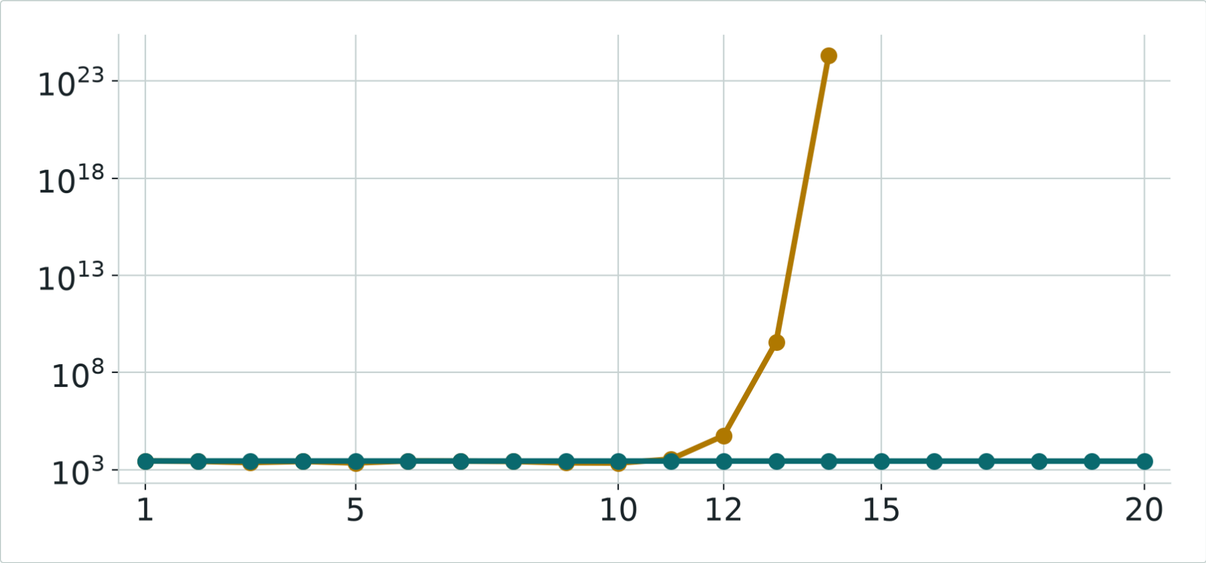
```
def 확인(self, 에폭: int, 검증_손실: float, 모델: nn.Module) -> bool:
    if 검증_손실 < self.최고_손실 - self.최소_개선:
        self.최고_손실 = 검증_손실
        self.최고_에폭 = 에폭
        self.최고_가중치 = copy.deepcopy(모델.state_dict())   # 복사해서 보관
        self.나빠진_횟수 = 0
    else:
        self.나빠진_횟수 += 1
    return self.나빠진_횟수 >= self.참을성
```

```
52에폭에서 멈춤 (최고 기록: 32에폭, 손실 0.4883)
멈춘 시점 검증 (손실, 정확도): [0.5369, 0.852]
되돌린 뒤 검증 (손실, 정확도): [0.4883, 0.896]
```

참을성 20, 최대 500에폭 · 멈춘 시점 85.2%를 **32에폭 가중치로 되돌려** 89.6%

14차시 · 8.8 조기 종료 · 예제 08-training-well/b15_early.py (핵심 부분, 2초)

8.9 이상치 하나가 학습을 터뜨린다



● 클리핑 없음 · ● max_norm=1.0 · 가로 단계 · 세로 손실

없음: 그래디언트 136 → 1만 → 4,500만
15단계 **inf**, 그 뒤로 **nan**

자르기: 200단계 동안
손실 2,817 → **2,016**

```
손실.backward()
크기 = nn.utils.clip_grad_norm_(모델.parameters(), max_norm=1.0)  # 자르기 전 크기를 돌려준다
옵티마이저.step()
```

14차시 · 8.9 그래디언트 클리핑 · 책의 b16_clip.py와 같은 데이터·모델로 다시 그린 그림 (처음 20단계)

8.10 재현성 체크리스트

```
def 씨앗_고정(씨앗: int = 42) -> None:
    random.seed(씨앗)
    np.random.seed(씨앗)
    torch.manual_seed(씨앗)
    os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
    torch.use_deterministic_algorithms(True)
```

```
씨앗 42, 첫 번째: 0.5159433484077454
씨앗 42, 두 번째: 0.5159433484077454
씨앗 7          : 0.5170934200286865
1번째 분할의 검증용 번호: [8, 6, 2]
2번째 분할의 검증용 번호: [8, 6, 2]
```

확인할 것	방법
난수 씨앗 셋	random.seed, np.random.seed, torch.manual_seed
분할·섞는 순서	random_split과 DataLoader에 씨앗 고정 generator=
결정적 연산	use_deterministic_algorithms(True), CUDA는 CUBLAS_WORKSPACE_CONFIG도
일꾼·환경	worker_init_fn으로 일꾼 씨앗, 파이썬·PyTorch 버전과 장치 기록

8.11 손실이 안 줄 때: 싼 확인부터

1

첫 손실

학습 전 손실이 2.30 근처인가

2

배치 하나

32장을 다 외우나

3

학습률 범위

알맞은 범위인가

4

그래디언트

층마다 흐르나

5

흔한 버그

오류 없는 버그 네 가지

하이퍼파라미터부터 이것저것 바꾸는 것이 **가장 시간이 많이 드는** 방법이다

8.11 1단계 첫 손실, 2단계 배치 하나

학습 전 손실

경우	손실
찍기 수준 -ln(1/10)	2.3026
정상 모델	2.3190
마지막 층 × 100	**7.1326**

훨씬 크면 모르면서 강하게 확신하는 것. 초기화와 입력 스케일을 의심

32장만 200단계 외우기

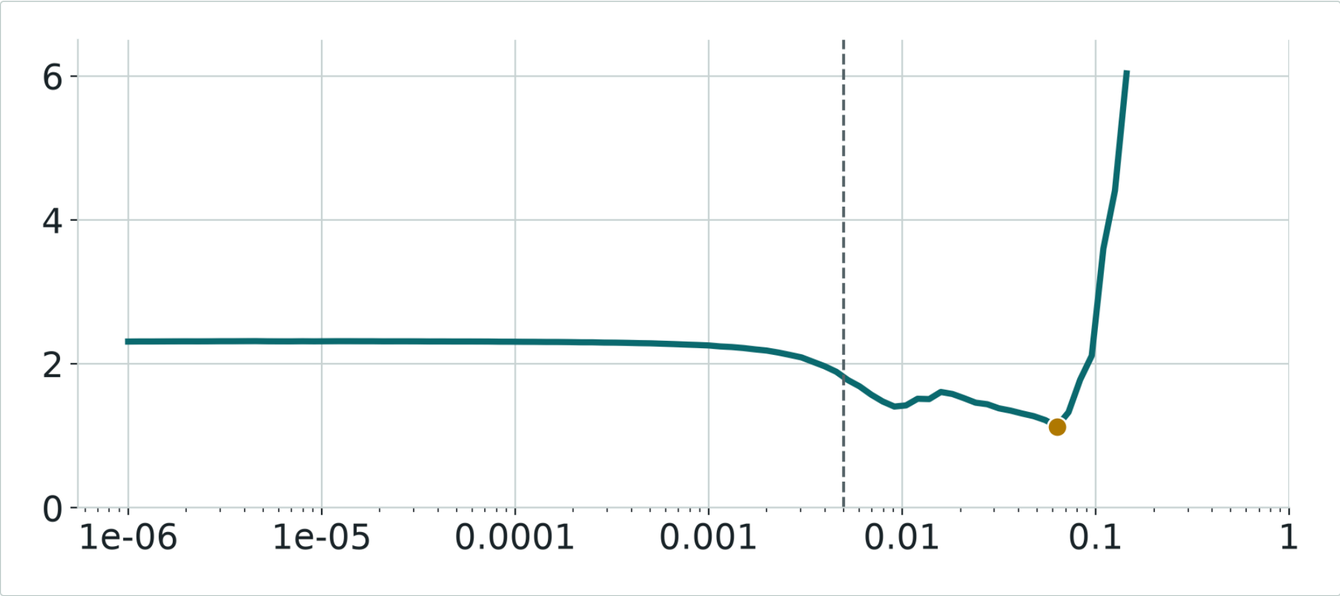
```
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
if 버그:
    모델 = 모델_만들기()  # 옵티마이저를 만든 뒤 다시 만들었다!
```

단계	0	50	100	200
정상	2.3140	0.0391	0.0035	**0.0011**
버그	2.2960	2.2960	2.2960	**2.2960**

32장조차 못 외운다면 데이터가 아니라 **코드** 문제 · 둘 다 몇 초면 끝난다

14차시 · 8.11 1단계·2단계 · 예제 08-training-well/b18_first_loss.py, b19_one_batch.py (출력을 표로)

8.11 3단계 학습률 범위 찾기



가로 학습률(로그) · 세로 손실(이동 평균) · ● 최저 · 점선 = 후보 0.005

1e-6에서 1까지 100단계 동안 같은 비율로 키우고, 손실이 최저의 4배를 넘으면 그만

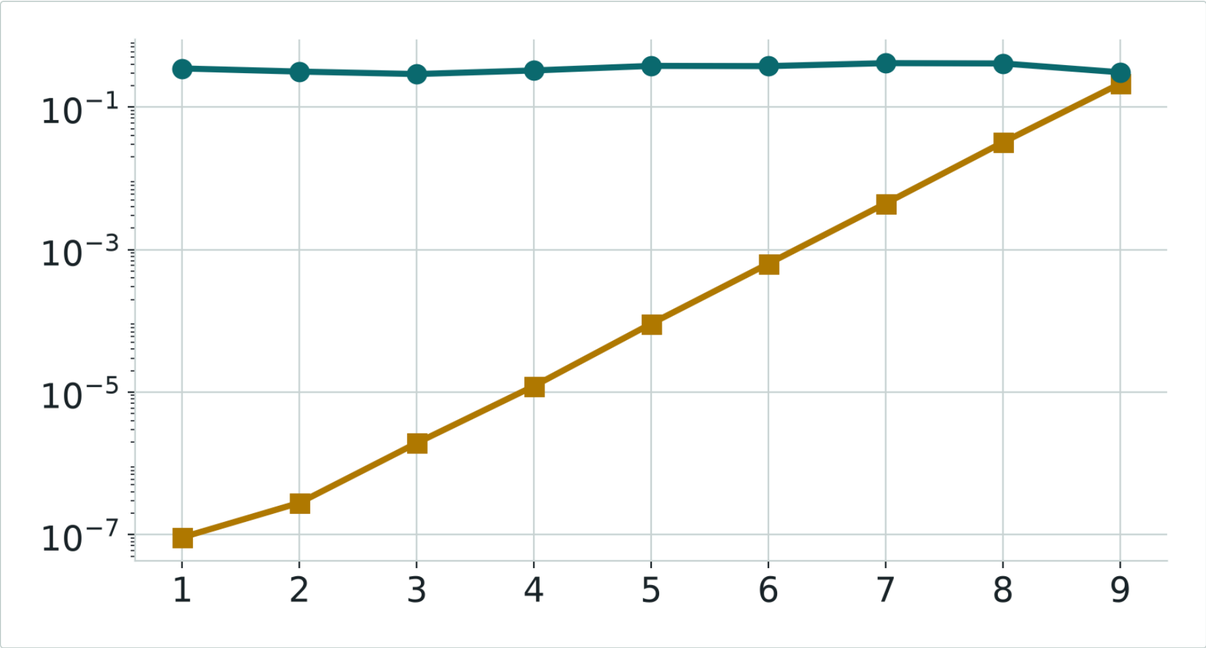
1e-4 아래: 거의 그대로

최저 6.31e-02에서 손실 1.1300

1.45e-01에서 6.0337로 폭발, 멈춤

최저의 10분의 1쯤을 고른다

8.11 4단계 그래디언트가 흐르는지 보기



■ Sigmoid · ● ReLU + Kaiming · 가로 층(1 = 첫 층)

```
for 이름, p in
  모델.named_parameters():
  p.grad.norm()
```

Sigmoid: 첫 층 9.2e-08
층마다 약 7분의 1씩 깎인다

ReLU + Kaiming:
모든 층 0.3~0.4 근처

앞쪽으로 갈수록 몇 자리씩 작아지면 **소실**, 터무니없이 크면 **폭주**, grad가 None이면 그래프가 끊긴 것

14차시 · 8.11 4단계: 그래디언트가 흐르는지 확인하기 · 책의 b21_grad_norm.py와 같은 모델로 다시 그린 그림

8.11 5단계 오류 없이 도는 흔한 버그

zero_grad() 누락

backward()는 더한다:
w.grad 3.0 → 6.0 → 9.0

검증 정확도
있음 83% → **98%**
없음 66%, 49%, 41% … **83%**

소프트맥스 두 번

300단계 학습 손실
한 번 0.0021 · 두 번 **1.5638**

넘을 수 없는 바닥 **1.4612**
검증 97.8% 대 89.4%

레이블 자료형·모양

실수 정답은 오류(Long이어야)

예측 (4, 1), 정답 (4,)이면
(4, 4)로 퍼져 손실 **2.8257**
진짜 손실은 3.7137

네 버그 중 나머지 하나 **eval() 누락**은 12차시에서 본 그대로: 82.8% 대 87.6%

14차시 · 8.11 5단계: 흔한 버그 목록 · 예제 08-training-well/b22_bug_zero_grad.py, b23_bug_softmax.py, b24_bug_dtype.py

8.11 증상으로 찾아보기

증상	의심할 것	확인·해결
손실이 처음부터 꼼짝 안 함	옵티마이저 연결, grad None	배치 하나, 층별 그래디언트
첫 손실이 ln(클래스 수)보다 큼	초기화, 입력 스케일	첫 손실, 입력을 0~1로
손실이 줄다가 nan	큰 학습률, 폭주, 이상치	학습률 범위, 클리핑
출렁이며 잘 안 줄어듦	zero_grad() 누락, 큰 학습률	학습 루프 순서
분류 손실이 1점대에서 멈춤	소프트맥스 두 번	모델 끝 소프트맥스 제거
평가할 때마다 성적이 다름	eval() 누락	평가 함수 첫 줄에 eval()
회귀 손실은 그럴듯한데 나쁨	예측·정답 모양 불일치	손실 전 shape 출력
앞쪽 층 그래디언트가 0 근처	그래디언트 소실	ReLU, Kaiming, 배치 정규화

직접 실행 10분

1. b15_early.py 복사본에서 참을성을 5와 50으로 바꿔 본다. 언제 멈추고, 되돌린 뒤 검증 정확도는?
2. b16_clip.py 복사본에서 max_norm을 10.0으로 키운다. 200단계 손실은?
3. b18_first_loss.py 복사본에서 100배를 10배로 바꾸면 첫 손실은 찍기 수준(2.30)에 얼마나 가까운가?

```
cd examples/vol2/08-training-well && uv run python b15_early.py
```

10개 분류의 학습 손실이 1.5 근처에서 더 내려가지 않는다. 먼저 의심할 것은?

Q

가 zero_grad() 누락

나 모델 끝의 소프트맥스

다 eval() 누락

라 테스트 데이터로 조기 종료

14차시 정리

조기 종료는 참을성만큼 기다리고 deepcopy로 보관한 최고 가중치로 되돌린다. 기준은 검증 데이터

클리핑은 backward()와 step() 사이. **재현성**은 씨앗 셋, 생성기, 결정적 연산, 환경 기록

디버깅은 첫 손실 → 배치 하나 → 학습률 범위 → 층별 그래디언트 → 버그 목록

다음 차시: 외우는 신경망에 대책을 하나씩 더하는 종합 실습

15차시

15

외우는 신경망에 대책을 하나씩

책 8.12절 ~ 8.13절, 과정 마무리

학습 목표

- 대책을 하나씩 더하며 검증 성적이 변하는 모습을 표로 읽는다
- 모든 대책이 늘 도움이 되지는 않는다는 것을 숫자로 본다
- 검증으로 고르고, 테스트로 마지막에 한 번 확인한다

8.12 실험 하나에 스위치 여섯 개

```
def 실험(씨앗: int, wd: float = 0.0, 드롭아웃: float = 0.0, 배치정규화: bool = False,
         조기종료: bool = False, 코사인: bool = False, 증강: bool = False):
    ...  # AdamW, 최대 150에폭, 조기 종료 참을성 20

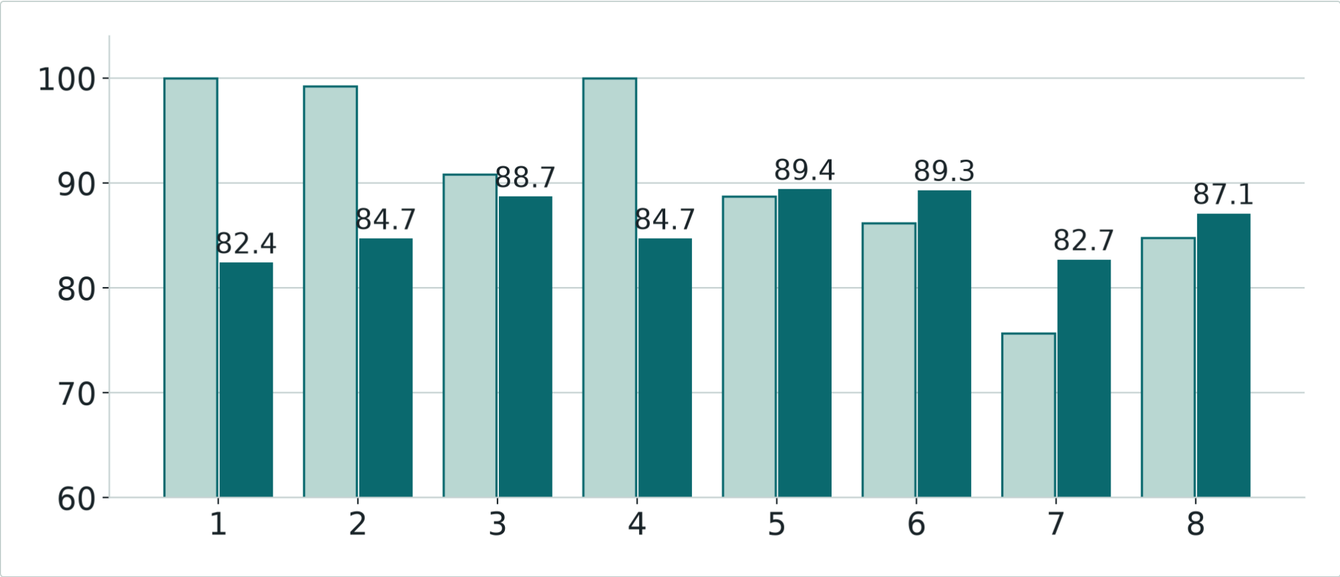
self.옮기기 = v2.RandomCrop(size=8, padding=1)  # 증강: 숫자를 최대 한 칸씩 옮긴다
```

단계	단계
1. 기준 (대책 없음)	5. 3번 + 조기 종료
2. + weight decay 1.0	6. 5번 + 코사인 스케줄러
3. + 드롭아웃 0.5	7. 5번 + 데이터 증강
4. + 배치 정규화	8. 기준 + 데이터 증강만

단계마다 **씨앗 0, 1, 2**로 세 번 학습해 평균 · 증강은 학습용 로더에만

15차시 · 8.12 실습: 외우는 신경망에 대책을 하나씩 더하기 · 예제 08-training-well/b25_lab.py (핵심 부분, 33초)

8.12 여덟 단계의 결과



옅은 막대 = 학습 정확도 · 진한 막대 = 검증 정확도(%) · 가로는 단계 번호 · 씨앗 셋 평균

단계	검증 손실	멈춘 에폭
1	0.732	150
2	0.494	150
3	0.459	150
4	0.478	150
5	0.459	128
6	0.482	126
7	0.777	134
8	0.504	150

가장 좋은 검증 정확도는 **5단계 89.4%**: weight decay 1.0 + 드롭아웃 0.5 + 조기 종료

15차시 · 8.12 결과 · 예제 08-training-well/b25_lab.py 출력(.out)의 숫자를 막대와 표로 옮긴 것

8.12 표에서 읽을 네 가지

학습이 내려가며 검증이 오른다

1~3단계 학습 100% → 90.8%, 검증 82.4% → 88.7%

늘 도움이 되지는 않는다

배치 정규화를 더하자 다시 100% 외우고 검증 84.7%

효과가 없으면 뺀다

코사인 스케줄러 89.3%, 5단계 89.4%와 차이 없음

증강은 혼자서도 세다

증강만 87.1%(+4.7%p), 5단계에 겹치면 82.7%로 과소적합

8.12 테스트 데이터로 마지막에 한 번

1. 기준 (대책 없음)

76.8%

검증 82.4%보다 5.6%p 낮다

5. 3번 + 조기 종료

86.5%

검증 89.4%보다 낮다

테스트 1,097장, 씨앗 셋 평균 · 대책의 효과는 같은 테스트 위의 차이 **약 10%p**로 읽고, 최종 실력은 테스트 점수로 보고한다

직접 실행 10분

1. b25_lab.py 복사본의 단계들을 아래 넷으로 바꾸고, 맨 끝의 테스트 부분은 지운다
2. 실행해(약 20초) 5단계보다 검증 정확도가 높은 설정이 있는지 찾는다

```
단계들 = [
    ("5. 3번 + 조기 종료", dict(wd=1.0, 드롭아웃=0.5, 조기종료=True)),
    ("5a. 드롭아웃 0.7로", dict(wd=1.0, 드롭아웃=0.7, 조기종료=True)),
    ("5b. weight decay 3.0으로", dict(wd=3.0, 드롭아웃=0.5, 조기종료=True)),
    ("8a. 증강 + 조기 종료", dict(증강=True, 조기종료=True)),
]
```

테스트 데이터는 고르는 데 쓰지 않는다: 다 고른 뒤에 한 번만

검증 정확도로 5단계를 골랐다. 이 모델의 실력으로 보고할 숫자는?

Q

가 검증 정확도 89.4%

나 테스트 정확도 86.5%

다 학습 정확도 88.7%

라 여덟 단계 검증의 평균

15차시 정리: 8장 한 장으로

진단이 먼저: 학습·검증 곡선을 함께 보고, 과적합할 만큼 학습시킨 뒤 정규화한다

대책은 골라 쓴다: weight decay, 드롭아웃, 조기 종료, 증강은 검증 데이터로 확인하며 하나씩

고르는 것은 검증, 보고는 테스트: 대책 없음 76.8% → 5단계 86.5%

다음: 3일 과정 돌아보기

3일 동안 만든 것: 학습 한 바퀴

1일 · 1~5차시

텐서와 기울기

숫자를 담는 그릇, autograd로 구한 기울기로 조금씩 고치기

2일 · 6~10차시

모델과 데이터

nn.Module과 표준 학습 루프, Dataset·DataLoader·transforms

3일 · 11~15차시

잘 학습시키기

파이프라인 마무리, 진단하고 대책을 골라 쓰고 디버깅하기

데이터를 나르고(7장), 모델을 만들고(6장), 기울기로 고치고(5장), 외우지 않게 지킨다(8장). 그릇은 모두 텐서(4장)

3일차에 배운 것

1. 길이를 **collate_fn**으로 맞추고, 씨앗 고정으로 나누고, 드문 클래스는 더 자주
2. 학습·검증 곡선으로 **진단**하고 weight decay, 드롭아웃으로 막는다
3. 깊은 모델은 **배치 정규화와 초기화**로 살리고, 학습률은 스케줄러로
4. 조기 종료, 클리핑, 재현성, 그리고 **싼 확인부터** 디버깅
5. 대책은 **검증으로 골라** 쓰고, 테스트로 한 번 확인

5일 과정: 내일 4일차 CNN

- 그림을 펴지 않고 받는 합성곱
- 패딩과 보폭, 채널과 풀링
- 첫 CNN, 밀어도 알아볼까
- BN·드롭아웃, 작은 ResNet 실습

3일 과정은 여기서 마칩니다